

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a2229615094e34b93.jsonl`  
- SHA-256: `9a24162c4446bf7ea62e160ed820e03f27d7a5ea89f075116ce2901432d22bae`  
- Source modified: `2026-07-02T04:56:52+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_7049e1ed-b93`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T04:45:39.418Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.

REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).

AUDIT FINDINGS YOU ARE FIXING: R1-03 (/api/source full re-download per Range request) ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.

TASK BRIEF:

In C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer: fix GET /api/source (backend/main.py ~:646 get_source) so a REMOTE-backed video is fetched ONCE and reused: cache the fetched local copy keyed by video_id under output_base(cfg) (e.g. <output>/source_cache/<video_id>/) with an atomic write (download to .partial then rename); subsequent requests (every browser Range request!) serve the cached file. Do NOT finally-delete (FileResponse streams after return ? verifier note). Handle concurrent first-requests safely (lock or tolerate double-fetch with atomic rename). Local-path videos stay identity passthrough (byte-identical behavior). Tests: remote ref fetched once across two requests (mock storage backend counting fetches), local passthrough unchanged, partial-file never served.

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):

1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.

COMMIT/PR CONVENTIONS:

- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authoried-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.

QUALITY BAR:

- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md \$0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.
- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.

- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:48.316Z)

master...origin/master

3. user (2026-07-02T04:45:49.616Z)

70:### R1-03 [P1/M] critical-fix ? badminton-highlight-indexer

4. user (2026-07-02T04:45:56.726Z)

```
1      # CLAUDE.md ? project memory / agent handoff
2
3      Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4      FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
5      repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7      ## Read these first
8      - **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
9        contracts (the cold-start map; read before touching code).
10     - **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
11     live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12     disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13     - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14     `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15     (offline-segmenter narrative) and `docs/BACKLOG.md`.
16     - **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**
17     `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18     (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19     "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`
(A/B + shadow
20     eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21     (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
22     - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23     **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move
24     **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
25     archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
26     note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
27     lifecycle register in **`docs/DOC_STATUS.md`**.
28
29     ## Current state (June 2026)
30     - **Scaffolding complete** ; strategy foundation + owned-model roadmap merged. **No
```

```

31     platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
32     sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
33     validation, reliability, CI + test coverage) ? see git history.
34     - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
35     (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
36     greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
37     implementation without explicit owner approval.**
38
39     ## Architecture in one breath
40     - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
41     / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
42     `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
43     - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
44     - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
45     user_version` migration ladder in `backend/infrastructure/database.py`.
46
47     ## Committed guardrails (non-negotiable)
48     - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49     training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50     code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51     - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52     is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
53     needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54     minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55     - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
56     (GDPR Art. 9).
57     - **Generic data schema** ; `coach ? athlete` is an **optional overlay**, not baked in.
58     - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/`;
59     Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60     on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61     DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
62     `deploy/digitalocean/` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64     ## Workflow conventions
65     - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
66     - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67     - Never push to the default branch directly; descriptive commit messages.
68     - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69     - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link
breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI

```

```
job (`python -m tools.check_md_links`).
70     - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
71     who has the golden corpus + labelled videos; verify pure-logic paths locally.
72
```

5. user (2026-07-02T04:45:56.894Z)

```
50     ## Confirmed findings ? round 1 (43)
51
52     ### R1-01 [P1/M] tech-debt ? badminton-highlight-indexer
53     **PR #390 (D1/P10 main.py split) is stale and CONFLICTING while main.py grew 52% since
it was cut**
54     - anchor: `backend/main.py:1` · tracked in: PR #390;
docs/CODE_CLEANUP_AUDIT_2026-06-24.md §6 row P10 · finder: engine:audit-doc-reconcile
55     - evidence: gh pr view 390: state=OPEN, mergeable=CONFLICTING, mergeStateStatus=DIRTY,
updatedAt=2026-06-26T14:37Z, 3 files (backend/main.py, backend/orchestration.py, mypy.ini).
Since then >=8 PRs modified backend/main.py (#400, #405-#409, #412, #413 per git log
--since=2026-06-25 -- backend/main.py). wc -l backend/main.py = 2198 vs the audit doc's '1448
lines' claim (docs/CODE_CLEANUP_AUDIT_2026-06-24.md:279); 18 @app. routes and 45 print() calls
now live in it; backend.main still mypy-quarantined (mypy.ini:15). No backend/orchestration.py
exists on master.
56     - why it matters: This is the last HIGH-severity item of the cleanup audit (P10, the
only non-Phase-3 ? row in its §6 tracker) and it is compounding: every new endpoint PR lands in
the god-module, widening the conflict surface and making the split more expensive each week. The
3-file extraction in #390 is now unmergeable.
57     - proposed fix: Close #390 and re-cut the orchestration.py extraction fresh from
origin/master (small, ~1 session), or rebase it now. Then continue the split (CLI handlers ->
backend/cli/, new /api/source|api/assets|api/capabilities routes -> backend/api/) before more
endpoints accrete. Update the audit doc's stale '~1450 lines' sizing while at it.
58     - verifier notes: Minor sharpenings only: (a) git log --since=2026-06-25 --
backend/main.py actually shows 10 commits (adds #387 repo-wide ruff format ? itself a likely
source of #390's conflicts ? and #380), so ">=8 PRs" is understated; (b) the mypy quarantine
block is mypy.ini:15-16 ([mypy-backend.main] header at 15, ignore_errors at 16); (c) the audit
doc's "1448 lines" statement is in the §Changelog 2026-06-25 "Accuracy fixes" bullet (~line
278), and the stale "~1450" sizing also appears at lines 69, 198, and 247 ? all four spots need
the size refresh the fix proposes.
59     - verifier reasoning: Every load-bearing claim reproduced. (1) `gh pr view 390` returns
state=OPEN, mergeable=CONFLICTING, mergeStateStatus=DIRTY, updatedAt=2026-06-26T14:37:18Z,
exactly 3 files: backend/main.py (+19/-595), backend/orchestration.py (ADDED, +635), mypy.ini
(+6). (2) Current working tree (branch master, HEAD e410136 = PR #413): `wc -l backend/main.py`
= 2198; 1448?2198 is +51.8% ? the claimed 52%; the 1448 baseline is documented in
docs/CODE_CLEANUP_AUDIT_2026-06-24.md changelog ("main.py size '~1300' ? '~1450' (now 1448
lines)", 2026-06-25 entry, ~line 278-279). (3) Not fixed: backend/orchestration.py does not
exist in the tree, and git log --since=2026-06-25 -- backend/main.py shows the file only grew ?
commits for PRs #413, #412, #409, #408, #407, #406, #405, #400 (the claimed 8; the log actually
lists 10 incl. #387 ruff-format and #380). (4) grep counts on backend/main.py: 18 lines matching
`^@app\.` and 45 `print(` occurrences ? exactly as claimed. (5) mypy.ini:15-16 = `[mypy-
backend.main]` / `ignore_errors = True` (quarantine intact). (6) Audit doc §6 tracker line 198:
P10 row "D1: Split main.py god-module (~1450 lines ? smaller modules)" is ?; the only other ?
row is line 205 (P16+ Phase 3 bucket), so P10 is indeed the sole non-Phase-3 open item; D1 is
rated HIGH at doc line 69. Severity P1 is honest for a tech-debt audit: the concrete failure is
already materialized (the 635-line extraction in #390 is unmergeable today), and it compounds ?
8 endpoint-adding PRs landed in the god-module in the week since the PR was cut, all inside a
mypy ignore_errors module, so each week of delay increases rework cost and unchecked-type
surface. Not owner-gated: internal refactor, no serving go-live / corpus-promotion / spend /
licensing implications ("P10" here is the audit tracker row, not Studio P10). Proposed fix
(close-and-recut or rebase from origin/master, then continue the split per the doc's own plan at
line 247, and fix the stale ~1450 sizing) is sane and correctly scoped; effort M is right given
the fix includes continuing the split beyond the re-cut.
60
61     ### R1-02 [P1/M] critical-fix ? badminton-highlight-indexer
62     **/api/process re-runs the paid Gemini segmenter for an already-processed video (re-bill
on refresh/'Try again') ? still unfixed at HEAD**
63     - anchor: `backend/main.py:1172` · tracked in: Khelsutra/badminton-highlight-indexer#340
```

(also #330) · finder: engine:correctness

64 - evidence: backend/main.py:1171-1172 computes video_id and `was\_reingest = db.get\_video(video\_id) is not None`, but was_reingest is only used for the report/destroy-after-confirm; there is NO reuse/skip branch ? the segmenter always runs (main.py:1251-1255). Verified against gh: issue #340 OPEN ('re-runs the full (paid) segmenter anyway'), plus #330 (MD5 segmentation reuse) OPEN.

65 - why it matters: A page refresh or SPA 'Try again' on an already-indexed video (same MD5) re-bills Gemini (~\$0.10/8-min match) and burns a multi-minute compute run for zero new information. Live-triggered once in the alpha (2026-06-22, per #340).

66 - proposed fix: In _execute_processing, when db.get_video(video_id) exists with status='processed' and existing play_segments (and the request doesn't explicitly force reprocess), return the stored segments as the job result instead of dispatching the segmenter; add a `force` flag to ProcessRequest for deliberate re-ingest.

67 - verifier notes: Severity P1 is honest: concrete failure scenario is a user-reachable one-click ("Try again" after the SPA's 10-min poll ceiling, live-triggered 2026-06-22 per #340) that silently re-bills a metered Gemini key ? and per #340 the M8 cost ledger does not meter Gemini yet, so the re-bill is invisible. Proposed fix is sane and correctly scoped: the skip condition must require status=="processed" AND existing play_segments (a prior "failed" record ? main.py:1196-1201 ? must still re-run), and the added `force` flag preserves the existing destroy-after-confirm re-ingest path (main.py:1233-1235, 1278). Not owner-gated: the fix reduces real cloud spend rather than incurring it, and the owner already specified the desired behavior + acceptance criteria in OPEN issue #340. Effort M is right (endpoint logic + ProcessRequest field + a no-Gemini-call regression test). One duplicate-tracking note: #330 covers the same reuse mechanism as an optimization; a fix should close both.

68 - verifier reasoning: Verified against origin/master at HEAD e410136 (pulled, "Already up to date"). (1) Code behaves as claimed: backend/main.py:1171-1172 computes `video\_id = compute\_video\_id(local\_video)` then `was\_reingest = db.get\_video(video\_id) is not None`; the ONLY other use of `was\_reingest` in `\_execute\_processing` is main.py:1327, where it is passed to `emit\_indexer\_report` as a report field. There is no reuse/skip/short-circuit branch: after validation passes (main.py:1210-1214 upserts status "processed"), the flow unconditionally dispatches segmentation ? cloud via `\_maybe\_dispatch\_remote` (main.py:1241-1248) or in-process `segmenter.process\_video` (main.py:1251-1255). ProcessRequest (main.py:313-324) has no `force`/`reprocess` field (only video_path, player_pool, max_frames, segmenter, locale, compute). (2) Not fixed: `git log --oneline -15 -- backend/main.py` shows only unrelated commits (#413, #412, #409, #408, #407, #406, #405, #400, ...); no idempotency/reuse change. (3) Cost claim is real in served config: config.json:26 sets `"default\_segmenter": "gemini"` (the paid path is the production default; the "motion" fallback at main.py:1181-1183 is only the code-level default). (4) Tracking verified via gh: issue #340 OPEN (title and body match the finding verbatim, including the 2026-06-22 live trigger and ~\$0.10/8-min cost) and #330 OPEN (MD5 segmentation reuse).

69

70 ### R1-03 [P1/M] critical-fix ? badminton-highlight-indexer

71 **/api/source re-downloads the ENTIRE remote video from the bucket on every HTTP request (each browser Range request) into a fresh, never-deleted temp dir**

72 - anchor: `backend/main.py:646` · tracked in: ? · finder: engine:correctness

73 - evidence: backend/main.py:644-651: get_source calls storage.acquire_local_input(rec['filepath'], ...) on EVERY request; storage/__init__.py:89-90: for a non-local ref this does `tempfile.mkdtemp(prefix='rally-input-')` + full fetch ? a new dir + full download per call, never removed. Proxy warming is explicitly local-only (main.py:655-657), so a remote-source video never gets a proxy. FileResponse under fastapi>=0.111 (pyproject.toml:28) serves Range ? a browser scrubbing a <video> issues many separate requests to the same URL.

74 - why it matters: Direct cloud-egress cost leak plus a disk-fill denial-of-service of the appliance, reachable through a supported flow (remote-registered assets + P8 annotate-in-UI).

75 - proposed fix: Cache the fetched local copy keyed by video_id (e.g. under <output>/proxies or a per-video workspace source/ dir) and reuse it across requests; extend proxy warming to remote sources; delete or reuse the scratch dir instead of mkdtemp-per-request.

76 - verifier notes: Path correction: the storage citation is backend/storage/__init__.py:89-90 (not storage/__init__.py). Sharper failure scenario: store a 4 GB match via POST /api/assets with an s3://gs:// medium (P4, #409), open the P8 annotate UI, scrub ~10 times ? ~40 GB bucket egress (~\$4-5 real cost) and ~40 GB of never-deleted rally-input-* temp dirs per session; each seek also stalls behind a full multi-GB download, so the feature is effectively unusable for remote sources. Additional color for the fix: the engine free-space guard (require_free_bytes, #407) is not applied to this download path, and the cached

copy should live under a guarded root; the finding's proposed fix (cache keyed by video_id, reuse across requests, extend proxy warming to remote) is sane and correctly scoped.

77 - verifier reasoning: backend/main.py:646 calls storage.acquire_local_input(rec["filepath"], ...) on every GET /api/source/{video_id} when no proxy exists (proxy check main.py:641-642). backend/storage/__init__.py:89-90: non-local refs get a fresh tempfile.mkdtemp(prefix="rally-input-") + full fetch per call; docstring (lines 80-82) admits the copy "relies on OS temp reclamation" (nonexistent on Windows %TEMP%), and a repo-wide grep for "rally-input" shows no cleanup path. Remote filepaths are a supported state: POST /api/assets registers the cloud stored_uri as filepath (main.py:838, 855; comment at main.py:1680 lists gs://, s3://, gdrive://). Proxy warming is explicitly local-only (main.py:655-657, docstring 628-629 "remote-source proxying is a follow-up"), so remote sources never escape the full-fetch path. Installed fastapi 0.136.1 / starlette 1.0.0 FileResponse honors Range, so a scrubbing <video> issues many independent requests, each a full-object download. git log -15 -- backend/main.py (through #413) shows no fix after #406 introduced this. Not owner-gated: the fix is plain engineering that reduces cloud spend.

78

79 ### R1-04 [P1/M] tech-debt ? badminton-highlight-indexer

80 **Import cycles: extracted API modules reach back into backend.main globals at call time; config layer imports pipeline layer**

81 - anchor: `backend/api/job\_worker.py:75` · tracked in: docs/CODE_CLEANUP_AUDIT_2026-06-24.md P5 note + D1/P10 follow-up (line 221) · finder: engine:structure

82 - evidence: AST scan of backend/ found 5 strongly-connected components. (1) backend.main <-> backend.api.job_worker <-> backend.api.uploads: job_worker.py:75,98,214,234,251 and uploads.py:47 do call-time 'import backend.main as _main' to read main.py's module globals db_instance/global_config (main.py:279-280); main.py:292 re-exports job_worker names back onto itself. Documented as a transitional shim (CODE_CLEANUP_AUDIT P5 note, line 221) but it makes main.py load-bearing for every extracted module and keeps backend.main AND backend.worker.__main__ mypy-quarantined (mypy.ini:15-23). (2) Layering inversion: backend/config/models.py:249 imports segmenter_registry from backend.pipeline.segmenters.base inside a validator, while base.py:15 (_normalize_config) imports AppConfig back from config.models ? config validation now depends on pipeline registration side effects. (3-5) eval-package cycles (golden_fixtures<->golden_real_fixtures, ablation<->ablation_pdf, annotations<->gt_loader) are lazy and explicitly documented (golden_fixtures.py:1005-1028) ? lower priority.

83 - why it matters:

84 - proposed fix: Kill the shim as the concrete step-2 of D1: move db_instance/global_config onto app.state (create_app already exists per P5) and pass config/db explicitly to job_worker/uploads, then lift backend.main and backend.worker.__main__ out of mypy quarantine. For the layering inversion, move the segmenter-name validation out of the pydantic validator into a startup check (backend/config/startup.py) so config/ no longer imports pipeline/.

85 - verifier notes: Mechanism correction to the evidence: job_worker's five call-time imports do NOT read db_instance/global_config ? job_worker already takes config/db as explicit params (job_worker.py:89,205,230,247; mypy.ini:14 notes it was lifted from quarantine) and resolves FUNCTION seams (ProcessRequest, _execute_compile, _execute_processing, JobWaiting, _arm_wake_timer, _get_executor, _MAX_DRAIN_WAKE_SEC) through the backend.main module object per the documented test-monkeypatch contract (job_worker.py:8-16, main.py:286-291). Only uploads.py:49 reads a main global (global_config). So the app.state migration for db/config is mostly about uploads + main.py itself; the harder remaining work is redesigning the monkeypatch seam contract and its tests ? effort is top-end M. Sharper failure scenario: any alternative entrypoint (new console_script, uvicorn factory) that builds the app without running main() leaves backend.main.global_config=None and uploads silently falls back to default upload dir/size limits (uploads.py:49-52), ignoring operator security config ? same class as the already-documented python -m backend.main incident (main.py:2188-2195) that hung every queued job. Layering inversion is slightly worse than claimed: the validator's lazy import (models.py:249) transitively executes segmenters/__init__.py:1-7, importing ALL concrete segmenters, so constructing AppConfig anywhere drags the full pipeline in; and SegmenterRegistry.register (base.py:66-70) re-enters AppConfig construction during registration with errors swallowed by base.py:19's bare except.

86 - verifier reasoning: Verified in current tree (HEAD e410136, pulled clean). Cycle 1: backend/main.py:292-302 imports backend.api.job_worker at module load (re-exports); job_worker.py:75,98,214,234,251 do call-time 'import backend.main as _main' (grep-confirmed all 5); uploads.py:47-49 call-time imports and reads _main.global_config; globals defined main.py:279-280, set at main.py:101-103 and 1981-1982; mypy.ini:15-16 quarantines backend.main,

:22-23 backend.worker.__main__. Cycle 2: config/models.py:244-256 validator imports segmenter_registry from pipeline/segmenters/base; base.py:14-18 imports AppConfig back in _normalize_config; importing base initializes segmenters/__init__.py:1-7 which imports every concrete segmenter (motion.py:243 etc. register as import side effects), so AppConfig construction depends on pipeline import health ? and register() (base.py:66-70) re-enters AppConfig via segmenter_cls(None, {}) with failures swallowed by base.py:19 bare except. Eval cycles lazy+documented (golden_fixtures.py:1008-1013). Not already fixed: git log for the four files shows only feature commits (#405-#413). Severity honest: the failure class already bit ? main.py:2188-2195 documents the python -m backend.main dual-module incident (backend.main.db_instance=None, drain crashed silently, jobs hung 'queued' forever), patched only for that entry path (main.py:2196-2198); uploads.py:49-52 getattr-defaults mean any entriypoint that skips main() silently runs uploads with default dir/limits. Tracked_in accurate: audit doc line 221 is verbatim the P5 shim note. Not owner-gated: only owner question is D1 phase sequencing, none of the listed gates. Fix sane: create_app exists (main.py:2184), handlers already use app.state (mypy.ini:12-13), backend/config/startup.py exists as the startup-check home.

87
88 ### R1-05 [P1/M] tech-debt ? badminton-highlight-indexer
89 **Path-jail rejects gs://,s3://,gdrive-api:// before scheme resolution ? bucket ingest impossible on the exposed alpha box (issue #327 still open)**
90 - anchor: `backend/main.py:1376` · tracked in: <https://github.com/Khelsutra/badminton-highlight-indexer/issues/327> · finder: engine:security-config
91 - evidence: main.py:1376 calls `validate\_input\_path(req.video\_path, allowed\_root=allowed\_root)` BEFORE `storage.resolve\_input\_ref` at main.py:1379; backend/utls/paths.py:45-55 `resolve\_under\_root` realpaths the raw URI so `gs://bucket/x.mp4` escapes the jail -> 400. Startup check startup.py:106-119 makes the jail mandatory when exposed (EXPOSED_NO_VIDEO_ROOT), so an exposed engine can never ingest from a bucket. gh: issue #327 OPEN (verified 2026-07-02), proposing scheme-aware validation.
92 - why it matters: Blocks the alpha CUJ (large-video ingest via gs:// instead of 500 MB browser uploads) and is a stale-open thread since before 06-26; the fix must be done carefully to avoid reopening the arbitrary-local-file hole the jail exists for.
93 - proposed fix: Implement #327 option (b): jail only after resolve_input_ref, applying resolve_under_root exclusively to backend=='local' refs; keep traversal/symlink/absolute rejection for local paths and add both allow (bucket URI accepted) and reject (local traversal still 400) tests, updating tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri.
94 - verifier notes: Minor corrections: (1) startup check path is backend/config/startup.py:106-119, not startup.py (lines exact). (2) The "500 MB browser uploads" figure in why_it_matters comes from issue #327's body; the repo's actual browser-upload cap is security.max_upload_mb default 4096 MB (backend/api/uploads.py:51, backend/config/models.py:339) ? 500 MB is the Gemini provider per-clip cap (backend/providers/gemini.py:23). Immateral to the finding (multi-hour matches exceed 4 GB anyway; docs/VIDEO_LOCALITY_MODEL.md:26 "The 10 GB problem is real"). (3) Fix scope additions: there is a second resolve call-site (CLI --video-path, see tests/test_api_endpoints.py:922) ? verify whether it needs the same scheme-aware treatment; ensure unknown schemes still ValueError?400 via resolve_input_ref as the sole scheme gate. (4) Caution, not a gate: the change relaxes a deliberately pinned security contract on the exposed box; #327 records the owner's explicit want, but the option (a) vs (b) choice and the PR should get owner sign-off before landing since it alters exposed-serving posture.
95 - verifier reasoning: Personally reproduced from current code. (1) Ordering: backend/main.py:1376 calls validate_input_path(req.video_path, allowed_root=allowed_root) BEFORE storage.resolve_input_ref at main.py:1379-1381; ValueError maps to HTTP 400 at main.py:1382-1383. (2) Mechanism: backend/utls/paths.py:58-68 routes any request with allowed_root set through resolve_under_root, which os.path.realpath()s the RAW string (paths.py:47) ? 'gs://bucket/x.mp4' realpaths to a cwd-relative path, commonpath != root (paths.py:50-54) ? ValueError ? 400. The code itself documents this at main.py:1387-1389: "a configured allowed_video_root (hosted mode) rejects a non-local URI as out-of-root". (3) Pinned by test: tests/test_api_endpoints.py:903-912 asserts gs://, s3://, gdrive:// all ? 400 when allowed_video_root is set ("Pins the security contract"). (4) Exposure mandate: backend/config/startup.py:106-119 emits LEVEL_ERROR EXPOSED_NO_VIDEO_ROOT when edge_auth_enabled and allowed_video_root unset ? so an exposed engine must have the jail, making bucket ingest impossible there. (5) Not fixed: git log -15 -- backend/main.py (head e410136, #413) shows no commit addressing this; behavior present in current tree. (6) gh issue view 327: state OPEN, created 2026-06-22, title/body match the finding exactly, including the owner's explicit want ("ingest large videos via gs:// on the hosted box") and options (a)/(b). Concrete failure:

exposed alpha box (edge_auth_enabled=true, allowed_video_root=/jail per boot requirement) receives POST /api/process {"video_path":"gs://bucket/match.mp4"} ? 400 "path ... escapes the allowed root"; only ingest path left is the capped browser upload (backend/api/uploads.py:51, security.max_upload_mb) ? blocks the tracked alpha large-video CUJ. Severity P1 honest (owner-wanted CUJ blocked, open since 2026-06-22, workaround capped/painful per docs/VIDEO_LOCALITY_MODEL.md:19-26). Proposed fix (option b: jail only backend=='local' after resolve_input_ref, keep null-byte check universal and traversal/symlink jail for local, add allow+reject tests, update the pinned test) matches the issue's own proposal and is correctly scoped; effort M is right.

96

97 ### R1-06 [P1/S] doc-rot ? khelsutra (avidullu/khelsutra)

98 **Docs claim the retired droplet alpha is LIVE (droplet deleted 2026-06-25)**

99 - anchor: `NEXT\_STEPS.md:12` · tracked in: partially in issue #97 (item 2: update DEPLOY_ALPHA.md); the NEXT_STEPS.md droplet claim is untracked · finder: khelsutra:issues-triage

100 - evidence: NEXT_STEPS.md:12-13 ('Where we are (updated 2026-07-01)') still says the site is 'live + browsable on the D0 droplet -> http://168.144.126.176:8787'; deploy/DEPLOY_ALPHA.md:3 says 'Status: LIVE (deploy live-verified on the droplet 2026-06-22)' and :29 'This runbook ships the split topology'; docs/LOCAL_APPLIANCE_ARCHITECTURE.md:176 'The alpha stand-up is now LIVE (2026-06-22)'. Project memory (MEMORY.md:63, session-handoff.md:19): droplet claude-do-rally-test/168.144.126.176 RETIRED+deleted 2026-06-25; alpha paused pending Cloud Run. nslookup shows api./app.khelsutra.guru still resolve via Cloudflare to a tunnel with no origin (pending CNAME/Access cleanup).

101 - why it matters:

102 - proposed fix: One doc PR: mark DEPLOY_ALPHA.md status RETIRED/PAUSED with a pointer to the Cloud Run plan, fix NEXT_STEPS.md 'Where we are' (droplet gone, site is CF-Workers-only), and correct LOCAL_APPLIANCE_ARCHITECTURE.md:176. Also note the single-origin repoint that superseded the split topology (issue #97). The 2026-07-01 refresh (#139) touched docs but kept the stale claim, so this will keep misleading ramp-up reads.

103 - verifier notes: Minor anchor correction: the 'Where we are (updated 2026-07-01)' header is NEXT_STEPS.md:5; the droplet-live claim spans lines 12-13 (the finding's line 12 anchor is correct for the URL text). Severity P1 stands: NEXT_STEPS.md's 2026-07-01 stamp is NEWER than the handoff's 2026-06-25 correction, so ramp-up readers reconciling the two will trust the wrong doc; worse, DEPLOY_ALPHA.md §1 (line 35) directs deploying the engine + secrets onto 168.144.126.176 ? an IP the owner no longer controls and D0 can recycle. Fix is sane, doc-only, and not owner-gated (it records already-made owner decisions: retirement 2026-06-25 + single-origin repoint per issue #97); one caution ? the fix should point to the Cloud Run move as 'paused pending' rather than declare a new serving decision, which would be owner-gated. Did not independently re-verify the finder's nslookup/DNS claim; the finding stands without it.

104 - verifier reasoning: Working tree at HEAD f9eea14: NEXT_STEPS.md:5 'Where we are (updated 2026-07-01)' + :12-13 'live + browsable on the D0 droplet -> http://168.144.126.176:8787 (systemd khelsutra-site)'; deploy/DEPLOY_ALPHA.md:3 'Status: LIVE (deploy live-verified on the droplet 2026-06-22...)' + :29 'This runbook ships the split topology'; docs/LOCAL_APPLIANCE_ARCHITECTURE.md:176 'The alpha stand-up is now LIVE (2026-06-22): the engine booted on the droplet (168.144.126.176)'. Ground truth: memory/khelsutra-site-droplet-parity.md:3,10-14 (droplet DELETED 2026-06-25, data archived, doctl delete done, monitor removed via PR #117), session-handoff.md:19 ('the prior LIVE alpha on the droplet is GONE ? alpha paused pending the Cloud-Run move'), MEMORY.md:63. Not fixed: git log shows the last commit touching all three files is 9223ba9 (#139, the 2026-07-01 refresh) and the stale text persists at HEAD. Tracking verified: gh issue 97 is OPEN, item 2 covers only DEPLOY_ALPHA.md (single-origin update); the NEXT_STEPS.md droplet claim is untracked. Issue #97 also confirms split topology was superseded by the 2026-06-22 single-origin repoint, so DEPLOY_ALPHA.md:29 is doubly stale.

105

106 ### R1-07 [P1/M] critical-fix ? khelsutra (avidullu/khelsutra)

107 **Issue #111 still fully valid: refresh loses the in-flight job; progress static for minutes**

108 - anchor: `web/src/hooks/useIngestJob.ts:117` · tracked in: khelsutra#111 · finder: khelsutra:issues-triage

109 - evidence: useIngestJob.ts:117-128 ? job_id lives only in the poll closure (no localStorage/URL persistence); App.tsx:71-73 seeds state only from ?video= (set on done, App.tsx:153). A refresh mid-job lands the user back on the ingest form while the server job keeps running. Progress is the coarse free-text job.progress (useIngestJob.ts:106); no heartbeat/animation beyond a spinner chip (IngestPanel.tsx:337-358).

110 - why it matters:

111 - proposed fix: SPA half is Claude-doable now: persist the active job_id (?job=<id> +

localStorage) and on load resume polling to the existing ?video= handoff; add elapsed-time/heartbeat copy. Engine half (granular 'segmenting chunk 3/7' from backend/pipeline/segmenters/gemini.py via the job-progress hook) is a small cross-repo change. Per the issue this is 'the most likely way an early tester loses a working job'.

112 - verifier notes: Failure scenario, sharpened from issue #111's field report: an ~8-min upload spends ~5 min in Gemini segmentation with near-static "processing" text; the tester refreshes (natural when nothing moves), lands on the ingest form with no trace of the running job, and either re-submits ? a duplicate job, potentially on paid cloud GPU via the compute picker ? or abandons believing it failed; the finished video only appears later in the library with no pointer back. Fix note: the backend is already refresh-safe (jobs persisted in SQLite), so the SPA half (persist ?job=/localStorage, resume polling into the existing ?video= handoff, heartbeat/elapsed-time copy) is standalone and Claude-doable now; only the granular-progress half touches the sibling badminton-highlight-indexer repo.

113 - verifier reasoning: useIngestJob.ts:117-128 ? job_id is a closure-local (`let jobId` at :117, used only by poll(:128)); grep of web/src confirms no localStorage/URL persistence of any job id (only annotate flag, wizard key, i18n lang). App.tsx:71-73 seeds videoId solely from ?video=, written only on job-done via loadVideo (App.tsx:150-154 ? useIngestJob.ts:96-98), so a mid-job refresh renders the ingest form (App.tsx:338-343) while the server job keeps running. Progress is coarse free-text job.progress (useIngestJob.ts:106) shown as a spinner chip (IngestPanel.tsx:337-348), no heartbeat/elapsed time. git log -15 on both files (latest #140, #138, #128?) shows no fix; issue khelsutra#111 is OPEN and documents the identical gaps from a live 2026-06-22 alpha run. Engine path backend/pipeline/segmenters/gemini.py exists in the sibling badminton-highlight-indexer repo as the finder scoped it. Not owner-gated: pure client resiliency, no go-live/corpus/licensing/spend/legal decision.

114

115 ### R1-08 [P1/S] tech-debt ? khelsutra (avidullu/khelsutra) ? SPA web/ + marketing site/
 116 **Engine's vendored SPA bundle is 8 feature-PRs stale ? shipped appliance UI lacks the entire storage/annotation wave**

117 - anchor: `badminton-highlight-indexer/backend/ui/ui\_version.json:3` . tracked in: ? .
 finder: khelsutra:code

118 - evidence: backend/ui/ui_version.json pins spa_commit ed33a515 built_at 2026-06-22T17:59:59Z, but khelsutra web/src has 8 newer PRs all merged 2026-07-01 (git log: f9eea14 #140 fps-in-library, fb2f561 #138 FFmpeg banner, 8667dbf #137 medium picker, cc06c32 #128 AnnotatePanel, 9e9df72 #127 persistent name, 926b53a #126 vendored annotator, 39c1349 #124 library, d0285cd #122 e2e harness). grep for 'video-library'/'library.title' in backend/ui/assets/index-ukuS9wxa.js = 0 hits. Meanwhile the engine ALREADY serves the matching endpoints: /api/assets (backend/main.py:727), /api/source (main.py:623), /api/annotations (main.py:663), and PRs #411-413 (e410136, 81a78bf) enriched /api/videos specifically for the new SPA library ? so a wheel install serves an API whose UI can't reach its own features. The API_VERSION skew guard doesn't fire (still 1, main.py:408); nothing detects content staleness.

119 - why it matters:

6. user (2026-07-02T04:46:05.038Z)

Already up to date.

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
 81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
 8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)

7. user (2026-07-02T04:46:05.167Z)

20:## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

8. user (2026-07-02T04:46:10.400Z)

20 ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
 21
 22 Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an existing
 23 **registry / extension point** (`?`) over a new top-level file.
 24
 25 | You're adding? | Put it in? | Notes |
 26 |---|---|---|
 27 | Backend / runtime code | `backend/<subsystem>/` |

```

`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are register a class in the relevant `?` registry, not a new module tree. |
28 | Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are imported widely (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29 | Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app
code) | scripts/ | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing
backend, add `scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30 | Ops / maintenance tool (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31 | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32 | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a 70%
coverage floor. Pure/offline + mocked (no GPU/WSL/network). |
33 | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
terminal-state (DONE/REJECTED) work moves to `docs/archives/`. |
34 | A throwaway repro / debug script | `scratch/` | Gitignored + excluded from
ruff/pytest. Never imported by app code. |
35 | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
manifest committed (provenance), weights gitignored. |
36 | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37 | Committed regression fixture (video-free golden) |
`eval_baselines/fixtures/<slug>/` | Tracked, LF-pinned (`gitattributes`). Synthetic Tier-0
(or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with
no video/GPU/DB. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m
backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated). |
38 | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | Gitignored.
Per-video isolation is the tracked PER_VIDEO_WORKSPACE.md project (P0 helper shipped #264;
supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`). |
39
40 Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do
not move):
41 `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) .
`run_all_tests.py` (CI: `ci.yml`) .
42 `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) .
`run_eval.py`
43 (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
44 `scripts/doctor.py` (it shadowed the build system); invoke via `python
scripts/doctor.py` or `indexer --setup`.
45 New standalone run-drivers go in scripts/, not the root.
46
47 ---
48
49 ## 1. PIPELINE
50
51 ### 1A. Runtime: video file -> highlight reel
52 ...
53 typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
54 @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
55 -> video_id = MD5(whole file) @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
56 -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
57 FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)

```

```

58         [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
59         -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
60         -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
61         -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
62         -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
63         [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
64         -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
65         -> select segment ids -> [(start,end)] (+ stitching padding)
66         -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
67         per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
68         ...
69         Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
70         ...
71         P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
72         -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
73         -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
74         -> P4 db.add_play_segment + db.link_candidate_to_segment
75         ...
76         Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
77         Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
78
79         ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
80         ...
81         WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
82         -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
83         -> WasbRunner.run_predict(video, out_dir)
84         stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
85         -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
86         -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
87         -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
88         ...
89         (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
90
91         ### 1C. Calibration / eval flow (NO pipeline run unless told)
92         ...
93         GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->
List[Interval]
94         DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval] stage
? {candidates, final}
95         -> metrics.match_intervals (greedy temporal-IOU) -> metrics.score -> Score
96         -> harness.evaluate -> EvalReport -> report_to_dict
97         -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
98         -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py
[CI gate]

```

```
99 Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,  
cross_validate} @backend/eval/calibration.py
```

9. user (2026-07-02T04:46:11.131Z)

```
600         return os.path.join(output_base(cfg), "proxies")
601
602
603     def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
604         """Background, best-effort: generate the annotation proxy ONCE so subsequent
605         /api/source loads
606         serve the lighter, seekable copy. Deduped; never blocks the request; the source
607         keeps serving if
608         it fails."""
609         with _proxy_lock:
610             if video_id in _proxy_inflight:
611                 return
612             _proxy_inflight.add(video_id)
613         try:
614             from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
615             if not needs_proxy(src_local):
616                 return # already light enough ? no proxy worth making
617             make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg),
618 f"{video_id}.mp4"))
619         except Exception as e: # noqa: BLE001 - warming is best-effort
620             logger.warning("proxy: warm failed for %s: %s", video_id, e)
621         finally:
622             with _proxy_lock:
623                 _proxy_inflight.discard(video_id)
624
625
626 @app.get("/api/source/{video_id}")
627 def get_source(video_id: str, request: Request):
628     """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
629     Serves a
630     frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
631     still keyed
632     to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
633     in the
634     background for next time ? non-blocking, so the first load is never delayed. Proxy
635     warming is
636     LOCAL-source only (remote-source proxying is a follow-up)."""
637     rec = request.app.state.db.get_video(video_id)
638     if not rec or not rec.get("filepath"):
639         raise HTTPException(status_code=404, detail="Video not found.")
640     cfg = request.app.state.config
641
642     # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
643     try:
644         proxy_name = safe_output_filename(f"{video_id}.mp4")
645         proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
646 _proxies_dir(cfg))
647     except ValueError:
648         proxy_path = ""
649     if proxy_path and os.path.isfile(proxy_path):
650         return FileResponse(proxy_path, media_type="video/mp4")
651
652     # 2) Otherwise resolve + serve the source.
653     try:
654         local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage",
655 None))
656         is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage",
657 None))
658     except ValueError as e: # unsupported scheme etc. ? a clean client error, never a
659         500
```

```

649         raise HTTPException(status_code=400, detail=str(e)) from e
650     except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
651         raise HTTPException(status_code=502, detail=f"Could not resolve the source
video: {e}") from e
652     if not os.path.isfile(local):
653         raise HTTPException(status_code=404, detail="Source video file not found.")
654
655     # 3) Warm a proxy in the background for next time (local source only).
656     if is_local:
657         threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
658
659     media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
660     return FileResponse(local, media_type=media)
661
662
663     @app.post("/api/annotations")
664     def post_annotations(req: AnnotationsRequest, request: Request):
665         """Persist finished rally labels as the annotator CSV (P8), written verbatim to
666         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5
``video_id`` ? so it joins
667         the video by content hash exactly as the training pipeline expects. Byte-compatible
with the VLC
668         plugin + browser extension. A later step (owner-gated) promotes labels into the
durable corpus.
669
670         Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
validated as a bare
671         filename so an untrusted value can't escape the labels dir (arbitrary write)."""
672         try:
673             edge_auth.resolve_tenant(request.headers, request.app.state.config)
674         except edge_auth.EdgeAuthError as e:
675             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
676
677         cfg = request.app.state.config
678         labels_dir = os.path.join(output_base(cfg), "labels")
679         try:
680             name = safe_output_filename(f"{req.video_id}.rallies.csv")
681         except ValueError as e:
682             raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
683         os.makedirs(labels_dir, exist_ok=True)
684         path = os.path.join(labels_dir, name)
685         try:
686             path = resolve_under_root(path, labels_dir)
687         except ValueError as e:
688             raise HTTPException(status_code=400, detail=str(e)) from e
689         with open(path, "w", encoding="utf-8", newline="") as f:
690             f.write(req.csv or "")
691         # Count CSV data rows (a line starting with an integer rally_number) for an honest
ack.
692         num_rows = sum(1 for ln in (req.csv or "").splitlines() if ln[:1].isdigit() and ", "
in ln)
693         logger.info("annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows,
path)
694         return {"status": "ok", "num_rows": num_rows, "csv_uri": f"labels/{name}"}
695
696
697     def _storage_settings(cfg) -> Dict[str, Any]:
698         """The storage config as a plain settings dict (NO secrets ? see StorageConfig)."""
699         sc = getattr(cfg, "storage", None)
700         if sc is None:
701             return {}
702         return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
703

```

```

704
705     def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
706         """The durable, MD5-keyed destination inside the chosen medium:
707         ``<medium>/<video_id><ext>``. """
708         return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
709
710     def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
711         """The LOCAL directory a ``put`` into this medium would write into (so a free-space
712         BLOCK can be enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
713         blocked).
714         ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount. """
715         if medium_ref.backend == "local":
716             return medium_ref.key or "."
717         if medium_ref.backend == "gdrive":
718             from backend.storage.capabilities import _gdrive_mount_root
719             root = _gdrive_mount_root(storage_settings)

```

10. user (2026-07-02T04:46:20.409Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13
14    from __future__ import annotations
15
16    import os
17    import tempfile
18    from typing import Any, Dict, Iterator, Optional
19
20    from backend.storage.base import StorageBackend
21    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
22
23    __all__ = [
24        "StorageBackend",
25        "StorageRef",
26        "StorageStat",
27        "parse_uri",
28        "resolve_input_uri",
29        "get_storage",
30        "fetch",
31        "put",
32        "list_uris",
33        "acquire_local_input",
34        "input_is_local",
35        "resolve_input_ref",
36    ]
37
38
39    def _storage_dict(storage_config: Any) -> Dict[str, Any]:

```

```

40     """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
41     if storage_config is None:
42         return {}
43     if hasattr(storage_config, "model_dump"):
44         return storage_config.model_dump() # type: ignore[no-any-return]
45     return dict(storage_config)
46
47
48     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
49         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
50         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
51         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
52
53         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
54         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
55         sc = _storage_dict(storage_config)
56         uri = resolve_input_uri(
57             raw,
58             input_backend=sc.get("input_backend", "local"),
59             input_root=sc.get("input_root", ""),
60         )
61         return parse_uri(uri)
62
63
64     def input_is_local(raw: str, storage_config: Any = None) -> bool:
65         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
66         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
67         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
68         ``resolve_input_ref``)."""
69         return resolve_input_ref(raw, storage_config).backend == "local"
70
71
72     def acquire_local_input(
73         raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
74     ) -> str:
75         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
76
77         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
78         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
79         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
80         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
81         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
82         worker's ``cmd_infer`` already ships."""
83         sc = _storage_dict(storage_config)
84         ref = resolve_input_ref(raw, sc)
85         if ref.backend == "local":
86             return (
87                 ref.key
88             ) # identity ? no temp, no copy, byte-identical to passing the path directly
89         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
90         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
91

```

```

92
93 def _backend_class(name: str):
94     """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
95     if name == "local":
96         from backend.storage.local import LocalStorage
97
98         return LocalStorage
99     if name == "s3":
100         from backend.storage.s3 import S3Storage
101
102         return S3Storage
103     if name == "gcs":
104         from backend.storage.gcs import GCSStorage
105
106         return GCSStorage
107     if name == "gdrive":
108         from backend.storage.gdrive import GDriveStorage
109
110         return GDriveStorage
111     if name == "gdrive-api":
112         from backend.storage.gdrive_api import GDriveApiStorage
113
114         return GDriveApiStorage
115     raise ValueError(
116         f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
117     )
118
119
120 def get_storage(
121     backend: Optional[str] = None,
122     *,
123     config: Optional[Dict[str, Any]] = None,
124     **overrides,
125 ) -> StorageBackend:
126     """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
127     selects the implementation; its per-backend settings come from ``config[backend]`` merged
128     with ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
129     selectors."""
130     cfg = config or {}
131     name = backend or cfg.get("backend", "local")
132     # Per-backend settings live under the backend name; tolerate the hyphen?underscore
133     # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
134     # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
135     settings: Dict[str, Any] = dict(
136         cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
137     )
138     settings.update(overrides)
139     return _backend_class(name)(**settings)
140
141 def fetch(
142     uri: str,
143     dst: "str | os.PathLike[str] | None" = None,
144     *,
145     config: Optional[Dict[str, Any]] = None,
146     overwrite: bool = False,
147 ) -> str:
148     """Resolve ``uri`` to a local file path. For ``local`` this is an identity
149     passthrough (no copy); for s3/gcs it downloads to ``dst``."""
150     ref = parse_uri(uri)

```



```

151         return get_storage(ref.backend, config=config).fetch_to_local(
152             ref, dst, overwrite=overwrite
153         )
154
155     def put(
156         local_path: "str | os.PathLike[str]",
157         uri: str,
158         *,
159         config: Optional[Dict[str, Any]] = None,
160         content_type: Optional[str] = None,
161     ) -> str:
162         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
163         ref = parse_uri(uri)
164         get_storage(ref.backend, config=config).put(
165             local_path, ref, content_type=content_type
166         )
167         return uri
168
169
170
171     def list_uris(
172         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
173     ) -> Iterator[str]:
174         """Yield the URI of every object under ``prefix_uri``."""
175         ref = parse_uri(prefix_uri)
176         for r in get_storage(ref.backend, config=config).list(ref):
177             yield r.uri
178

```

11. user (2026-07-02T04:46:26.233Z)

Found 1 file

tests\test_api_annotations.py

12. user (2026-07-02T04:46:27.683Z)

```

backend\utils\paths.py:7: def output_base(config: Any) -> str:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-8-
"""The configured base dir for a segmenter's LOCAL scratch/staging artifacts
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-9-
(COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``) so the
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-10-
chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` / ``output.output_dir`` instead
of
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-11-
a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior literal.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-12-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-13-
This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which may be a
backend\utils\paths.py-14-    ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig
or the legacy dict-shim
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-15-
(``.get("output", {})``.get("output_dir")``); any miss or empty value falls back to
``"output"``.
backend\utils\paths.py-16-    try:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\utils\paths.py-17-
out = (
--
backend\utils\freespace.py:49: def require_free_bytes(
backend\utils\freespace.py-50-     path: str, need: Optional[int], *, stage: str = "write"
backend\utils\freespace.py-51-) -> None:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py-52-    """Raise HTTP 507 if ``path``'s filesystem can't hold
``need`` bytes; otherwise return.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-

```

```

indexer\backend\utils\freespace.py-53-
backend\utils\freespace.py-54-     ``need`` of ``None`` or ``<= 0`` is a no-op (nothing to
reserve). Measuring is best-effort: an
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py-55-     undeterminable free-space read logs a warning and
proceeds (we never deny on inability to
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py-56-     measure). A confirmed shortfall raises so the caller
fails fast with an actionable 507."""
backend\utils\freespace.py-57-     if not need or need <= 0:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py-58-         return
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\backend\utils\freespace.py-59-     free = free_bytes(path)

```

13. user (2026-07-02T04:46:32.425Z)

```

2:GET /api/source/{video_id} (serve a video's source for annotation) and
28:# ---- GET /api/source/{video_id} ----
35:     r = client.get("/api/source/abc123")
43:     assert client.get("/api/source/nope").status_code == 404
49:     assert client.get("/api/source/gone").status_code == 404
62:     r = client.get("/api/source/vidp")

```

14. user (2026-07-02T04:46:39.052Z)

```

1     """TestClient coverage for the P8 annotate-in-UI engine endpoints:
2     GET /api/source/{video_id} (serve a video's source for annotation) and
3     POST /api/annotations (persist finished rally labels as the byte-verbatim annotator
CSV)."""
4
5     import pytest
6
7     pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
8     from fastapi.testclient import TestClient
9
10    import backend.main as m
11    from backend.config.models import AppConfig
12    from backend.infrastructure.database import Database
13
14
15    class _InlineExecutor:
16        def submit(self, fn, *args, **kwargs):
17            fn(*args, **kwargs)
18
19
20    def _client(tmp_path, monkeypatch):
21        db = Database(str(tmp_path / "t.db"))
22        cfg = AppConfig()
23        cfg.output.output_dir = str(tmp_path) # labels land under tmp, not CWD/output
24        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
25        return TestClient(m.create_app(cfg, db)), db
26
27
28    # ---- GET /api/source/{video_id} ----
29    def test_source_streams_the_recorded_video(tmp_path, monkeypatch):
30        client, db = _client(tmp_path, monkeypatch)
31        vid = tmp_path / "match.mp4"
32        vid.write_bytes(b"fake-mp4-bytes-\x00\x01\x02")
33        db.add_video("abc123", str(vid), "processed", [])
34
35        r = client.get("/api/source/abc123")
36        assert r.status_code == 200
37        assert r.headers["content-type"].startswith("video/")
38        assert r.content == vid.read_bytes()

```

```

39
40
41 def test_source_404_for_unknown_video(tmp_path, monkeypatch):
42     client, _ = _client(tmp_path, monkeypatch)
43     assert client.get("/api/source/nope").status_code == 404
44
45
46 def test_source_404_when_the_file_is_gone(tmp_path, monkeypatch):
47     client, db = _client(tmp_path, monkeypatch)
48     db.add_video("gone", str(tmp_path / "not-there.mp4"), "processed", [])
49     assert client.get("/api/source/gone").status_code == 404
50
51
52 def test_source_serves_the_proxy_when_one_is_ready(tmp_path, monkeypatch):
53     """D4: a ready frame-preserving proxy is served instead of the (heavier) source."""
54     client, db = _client(tmp_path, monkeypatch)
55     src = tmp_path / "match.mp4"
56     src.write_bytes(b"SOURCE-bytes")
57     db.add_video("vidp", str(src), "processed", [])
58     proxies = tmp_path / "proxies"
59     proxies.mkdir()
60     (proxies / "vidp.mp4").write_bytes(b"PROXY-bytes") # a ready proxy (output_dir ==
tmp_path)
61
62     r = client.get("/api/source/vidp")
63     assert r.status_code == 200
64     assert r.content == b"PROXY-bytes" # proxy wins over source
65
66
67 # ---- POST /api/annotations ----
68 def test_annotations_persist_verbatim_csv(tmp_path, monkeypatch):
69     client, _ = _client(tmp_path, monkeypatch)
70     csv = (
71         "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
72         "1,8.800,11.500,winner,badminton,9\n"
73         "2,24.389,46.589,unforced_error,badminton,21\n"
74     )
75     r = client.post("/api/annotations", json={"video_id": "abc123", "csv": csv})
76     assert r.status_code == 200
77     body = r.json()
78     assert body == {"status": "ok", "num_rows": 2, "csv_uri":
"labels/abc123.rallies.csv"}
79
80     written = tmp_path / "labels" / "abc123.rallies.csv"
81     assert written.read_bytes() == csv.encode("utf-8") # byte-verbatim (pipeline join
needs it exact)
82
83
84 def test_annotations_reject_unsafe_video_id(tmp_path, monkeypatch):
85     client, _ = _client(tmp_path, monkeypatch)
86     r = client.post("/api/annotations", json={"video_id": "../evil", "csv": "x"})
87     assert r.status_code == 400
88     assert not (tmp_path.parent / "evil.rallies.csv").exists() # nothing escaped the
labels dir
89

```

15. user (2026-07-02T04:46:47.967Z)

```

backend\config\models.py:378: # scratch dir before processing (see
``backend.storage.acquire_local_input``).
backend\main.py:646: local = storage.acquire_local_input(rec["filepath"], getattr(cfg,
"storage", None))
backend\main.py:791: local_working = storage.acquire_local_input(
backend\main.py:1168: local_video = storage.acquire_local_input(
backend\main.py:1683: local_src = storage.acquire_local_input(

```

```

backend\main.py:2056:         local_video = storage.acquire_local_input(args.video_path,
storage_cfg)
tests\test_api_cloud_dispatch.py:114:         m.storage, "acquire_local_input", lambda path,
scfg: str(fake_local)
tests\test_api_cloud_dispatch.py:381:         m.storage, "acquire_local_input", lambda path,
scfg: str(fake_local)
tests\test_api_cloud_dispatch.py:556:         m.storage, "acquire_local_input", lambda path,
scfg: str(fake_local)
tests\test_compute_backend.py:203:         """BLOCKER guard: GCS/S3 fetch_to_local REQUIRES a dst (a
None dst raises) ? _read_json must
backend\storage\base.py:19:         """Backend-agnostic object storage. ``fetch_to_local`` + ``put``
are the two the
backend\storage\base.py:23:         def fetch_to_local(
backend\storage\base.py:61:         ``NotImplementedError`` as expected and fall back to
``fetch_to_local``."""
backend\storage\gdrive.py:123:         def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gdrive.py:127:         return self._local.fetch_to_local(
backend\storage\gcs.py:40:         def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py:42:         raise ValueError("GCSStorage.fetch_to_local requires a dst
path")
docs\COMPUTE_DECOUPLED_SERVING\README.md:161:- 2026-06-21 ? **M2 landed**
([#280](https://github.com/Khelsutra/badminton-highlight-indexer/pull/280)):
`input_backend`/`input_root` + URI-aware main API/CLI input via
`resolve_input_uri`/`acquire_local_input` (local = identity passthrough, byte-for-byte
unchanged; bucket/Drive fetched to scratch). Unsupported scheme ? clean 400; `local://` accepted
on the resolved key; hosted-mode rejects non-local URIs; source URI kept for DB/report
provenance. Suite 1105 green, cov 84.84%.
docs\COMPUTE_DECOUPLED_SERVING\README.md:172:[Omitted long matching line]
backend\storage\local.py:3:Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an
identity passthrough (the
backend\storage\local.py:30:         def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\s3.py:47:         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\s3.py:49:         raise ValueError("S3Storage.fetch_to_local requires a dst
path")
backend\storage\gdrive_api.py:132:         def fetch_to_local(self, ref, dst=None, *,
overwrite=False) -> str:
backend\storage\gdrive_api.py:135:         "GDriveApiStorage.fetch_to_local requires a
dst path (no local mount)"
backend\storage\__init__.py:33:         "acquire_local_input",
backend\storage\__init__.py:72: def acquire_local_input(
backend\storage\__init__.py:151:         return get_storage(ref.backend,
config=config).fetch_to_local(
docs\PACKAGING_PLAN.md:154:| **CUJ-2** In-UI `gs://` bucket ingest | DONE |
`acquire_local_input` + gcs backend already accept `gs://`; preserved when the user opts into a
cloud storage backend (+ ADC). |
tests\test_input_source.py:2:acquire_local_input.
tests\test_input_source.py:19:from backend.storage import acquire_local_input, input_is_local,
resolve_input_ref
tests\test_input_source.py:124:#
----- acquire_local_input
tests\test_input_source.py:125: def test_acquire_local_input_local_identity(tmp_path):
tests\test_input_source.py:129:     assert acquire_local_input(str(p)) == str(p)
tests\test_input_source.py:132: def test_acquire_local_input_relative_local_identity():
tests\test_input_source.py:133:     assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
tests\test_input_source.py:149: def
test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
tests\test_input_source.py:152:     out = acquire_local_input(
tests\test_input_source.py:162: def
test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
tests\test_input_source.py:165:     acquire_local_input("x.mp4", {"input_root":
"gs://bucket/videos"})
tests\test_input_source.py:169: def

```

```

test_acquire_local_input_never_fetches_for_local(monkeypatch):
tests\test_input_source.py:174:    assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
tests\test_input_source.py:176:    acquire_local_input("clip.mp4", {"input_backend":
"local", "input_root": ""})
tests\test_nightly_reelcount.py:250: def test_fetch_to_local_passthrough_vs_remote_dst(tmp_path,
monkeypatch):
tests\test_storage.py:147:    be.fetch_to_local(
tests\test_storage.py:210:    assert be.fetch_to_local(ref, str(dst)) == str(dst)
tests\test_storage.py:281:    be.fetch_to_local(ref, str(dst))
tests\test_storage.py:312:    assert os.path.samefile(be.fetch_to_local(ref), root / "Sharing" /
"clip.mp4")
tests\test_storage.py:315:    assert be.fetch_to_local(ref, str(dst)) == str(dst)
tests\test_storage.py:363:    # A mounted Drive file has no presigned URL; callers fall back to
fetch_to_local.
tests\test_storage_gdrive_api.py:155:    be.fetch_to_local(ref, str(dst))
tests\test_storage_gdrive_api.py:178:    be.fetch_to_local(ref, str(dst))
tests\test_storage_gdrive_api.py:186:    be.fetch_to_local(
tests\test_storage_gdrive_api.py:193:    be.fetch_to_local(parse_uri("gdrive-
api://a/b.mp4"), None)

```

16. user (2026-07-02T04:46:48.139Z)

```

1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2
3      import os
4      from typing import Any, Optional
5
6
7      def output_base(config: Any) -> str:
8          """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
9             (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
10         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
11         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
12
13         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
14         ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim
15         (``.get("output", {}).__get__("output_dir")``); any miss or empty value falls back to
``"output"``. """
16         try:
17             out = (
18                 config.output.output_dir
19                 if not isinstance(config, dict)
20                 else config.get("output", {}).__get__("output_dir")
21             )
22         except Exception:
23             out = None
24         return out or "output"
25
26
27     def safe_output_filename(name: str) -> str:
28         """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
29
30         ``os.path.join(output_dir, name)`` silently discards ``output_dir`` when ``name`` is
absolute,
31         and ``..`` segments escape it ? so an unchecked ``output_filename`` is an arbitrary-
write
32         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
33         """
34         if not name or not name.strip():

```

```

35         raise ValueError("output_filename must not be empty")
36     if "\x00" in name:
37         raise ValueError("output_filename must not contain a null byte")
38     if "/" in name or "\\" in name:
39         raise ValueError(f"output_filename must not contain a path separator: {name!r}")
40     if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
41         raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
42     return name
43
44
45     def resolve_under_root(path: str, root: str) -> str:
46         """Return the real path, asserting it is contained under `root` (raises ValueError
47         if not)."""
48         rp = os.path.realpath(path)
49         rr = os.path.realpath(root)
50         try:
51             contained = os.path.commonpath([rp, rr]) == rr
52         except ValueError: # different drives on Windows
53             contained = False
54         if not contained:
55             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
56         return rp
57
58     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
59         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
60
61         With no allowed_root (default, single-user local mode) only the null-byte check
62         applies,
63         so the local 'point it at any of my videos' workflow is preserved. Setting an
64         allowed_root (a future hosted/tenant deployment) enforces containment.
65         """
66         if "\x00" in path:
67             raise ValueError("video_path must not contain a null byte")
68         if allowed_root:
69             resolve_under_root(path, allowed_root)

```

17. user (2026-07-02T04:46:54.256Z)

```

120     with pytest.raises(ValueError):
121         resolve_input_ref("https://host/x.mp4")
122
123
124     # ----- acquire_local_input
125     def test_acquire_local_input_local_identity(tmp_path):
126         p = tmp_path / "v.mp4"
127         p.write_bytes(b"abc")
128         # Identity passthrough ? returns the SAME string, no copy, no temp dir.
129         assert acquire_local_input(str(p)) == str(p)
130
131
132     def test_acquire_local_input_relative_local_identity():
133         assert acquire_local_input("rel/clip.mp4") == "rel/clip.mp4"
134
135
136     def _capturing_fetch(captured):
137         def _f(uri, dst=None, *, config=None, overwrite=False):
138             captured["uri"] = uri
139             captured["dst"] = dst
140             captured["config"] = config
141             if dst:
142                 with open(dst, "wb") as fh:
143                     fh.write(b"remote-bytes")
144             return dst or uri

```

```

145
146     return _f
147
148
149 def test_acquire_local_input_remote_fetches_to_scratch(monkeypatch):
150     captured = {}
151     monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
152     out = acquire_local_input(
153         "gs://bucket/videos/x.mp4", {"backend": "local", "gcs": {}}
154     )
155     assert captured["uri"] == "gs://bucket/videos/x.mp4" # exact URI routed to fetch
156     assert os.path.basename(out) == "x.mp4" # named from the ref
157     assert os.path.isfile(out) and open(out, "rb").read() == b"remote-bytes"
158     # The whole storage settings dict is forwarded so per-backend (gcs/s3) config
reaches fetch.
159     assert captured["config"] == {"backend": "local", "gcs": {}}
160
161
162 def test_acquire_local_input_input_root_joins_then_fetches(monkeypatch):
163     captured = {}
164     monkeypatch.setattr(storage, "fetch", _capturing_fetch(captured))
165     acquire_local_input("x.mp4", {"input_root": "gs://bucket/videos"})
166     assert captured["uri"] == "gs://bucket/videos/x.mp4" # resolved BEFORE fetch
167
168
169 def test_acquire_local_input_never_fetches_for_local(monkeypatch):
170     def _boom(*a, **k):
171         raise AssertionError("fetch must not be called for a local input")
172
173     monkeypatch.setattr(storage, "fetch", _boom)
174     assert acquire_local_input("/data/x.mp4") == "/data/x.mp4"
175     assert (
176         acquire_local_input("clip.mp4", {"input_backend": "local", "input_root": ""})
177         == "clip.mp4"
178     )
179

```

18. user (2026-07-02T04:46:55.772Z)

```

595:_proxy_inflight: "set[str]" = set()
596:_proxy_lock = threading.Lock()
599:def _proxies_dir(cfg) -> str:
607:    with _proxy_lock:
608:        if video_id in _proxy_inflight:
609:            _proxy_inflight.add(video_id)
610:        make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4"))
619:        with _proxy_lock:
620:            _proxy_inflight.discard(video_id)
638:        proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
_proxies_dir(cfg))

```

19. user (2026-07-02T04:48:00.669Z)

```

1     """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3     Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4     live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5     pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6     no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7     """
8
9     from __future__ import annotations

```

```

10
11 import os
12 from abc import ABC, abstractmethod
13 from typing import Iterator
14
15 from backend.storage.ref import StorageRef, StorageStat
16
17
18 class StorageBackend(ABC):
19     """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
20     worker dispatcher needs; the rest support listing/checkpointing/serving."""
21
22     @abstractmethod
23     def fetch_to_local(
24         self,
25         ref: StorageRef,
26         dst: "str | os.PathLike[str] | None" = None,
27         *,
28         overwrite: bool = False,
29     ) -> str:
30         """Make the object available as a local file; return its local path. For the
31         ``local`` backend this is an identity passthrough (the object is already
32         local)."""
33
34     @abstractmethod
35     def put(
36         self,
37         local_path: "str | os.PathLike[str]",
38         ref: StorageRef,
39         *,
40         content_type: "str | None" = None,
41     ) -> StorageRef:
42         """Upload/copy a local file to ``ref``; return ``ref``."""
43
44     @abstractmethod
45     def exists(self, ref: StorageRef) -> bool: ...
46
47     @abstractmethod
48     def stat(self, ref: StorageRef) -> StorageStat: ...
49
50     @abstractmethod
51     def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
52         """Yield a ``StorageRef`` per object under ``prefix``."""
53
54     @abstractmethod
55     def delete(self, ref: StorageRef) -> None: ...
56
57     def signed_url(
58         self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET"
59     ) -> str:
60         """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
61         sign (``local`` cannot; Drive only emulates) ? callers must treat
62         ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
63         raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")

```

20. user (2026-07-02T04:48:01.060Z)

```

1     """Free-space guards for the local box (STUDIO_MEDIA_LIFECYCLE.md P2, D11).
2
3     The single-box appliance fills its disk in three peaks: a chunked **upload** (raw file
4     into
5     ``upload_dir``), a **fetch-to-scratch** of a bucket/Drive source (a full local copy
6     under the temp
7     dir), and ? the real balloon ? **WSL frame extraction**, which can materialise ~15× the

```



```

source in
6     PNGs. D11: BLOCK on a local/WSL shortfall (buckets are warn-only elsewhere ? they aren't
7     capacity-bounded from here).
8
9     ``require_free_bytes`` is the HTTP-boundary guard: it raises **HTTP 507 (Insufficient
Storage)**
10    when the filesystem holding ``path`` can't hold ``need`` bytes, so the client fails fast
instead of
11    crashing a job mid-flight. It is best-effort about *measuring* (an unreadable
``disk_usage`` never
12    blocks ? inability to measure must not deny a legitimate request), but strict once a
real shortfall
13    is seen. The WSL frame-stage guard lives on ``wslCommandMixin`` (a different filesystem,
measured via
14    ``df`` inside WSL, not ``shutil.disk_usage``) ? see ``pipeline/detectors/base.py``.
15    """
16
17    from __future__ import annotations
18
19    import logging
20    import shutil
21    from typing import Optional
22
23    from fastapi import HTTPException
24
25    logger = logging.getLogger(__name__)
26
27
28    def free_bytes(path: str) -> Optional[int]:
29        """Best-effort free bytes on the filesystem holding ``path`` (``None`` if
undeterminable).
30
31        Never raises. ``shutil.disk_usage`` wants an existing path; we walk up to the
nearest existing
32        ancestor (an ``upload_dir``/scratch target may not exist yet at guard time)."""
33        import os.path
34
35        probe = path
36        for _ in range(64): # bounded ancestor walk (defensive against a pathological path)
37            if not probe:
38                return None
39            try:
40                return int(shutil.disk_usage(probe).free)
41            except (OSError, ValueError):
42                parent = os.path.dirname(probe.rstrip("/\\"))
43                if parent == probe: # reached the root and still can't stat ? give up
44                    return None
45                probe = parent
46        return None
47
48
49    def require_free_bytes(
50        path: str, need: Optional[int], *, stage: str = "write"
51    ) -> None:
52        """Raise HTTP 507 if ``path``'s filesystem can't hold ``need`` bytes; otherwise
return.
53
54        ``need`` of ``None`` or ``<= 0`` is a no-op (nothing to reserve). Measuring is best-
effort: an
55        undeterminable free-space read logs a warning and proceeds (we never deny on
inability to
56        measure). A confirmed shortfall raises so the caller fails fast with an actionable
507."""
57        if not need or need <= 0:
58            return

```

```

59         free = free_bytes(path)
60         if free is None:
61             logger.warning(
62                 "free-space guard (%s): could not read free space at %s ? proceeding.",
63                 stage,
64                 path,
65             )
66             return
67         if free < need:
68             need_mb = need / (1024 * 1024)
69             free_mb = free / (1024 * 1024)
70             logger.warning(
71                 "free-space guard (%s): %s needs %.0f MB but only %.0f MB free ? refusing
(507).",
72                 stage,
73                 path,
74                 need_mb,
75                 free_mb,
76             )
77             raise HTTPException(
78                 status_code=507,
79                 detail=(
80                     f"Not enough free disk space for {stage}: need ~{need_mb:.0f} MB, "
81                     f"only ~{free_mb:.0f} MB free. Free some space and retry."
82                 ),
83             )
84

```

21. user (2026-07-02T04:48:08.265Z)

```

1:import argparse
2:import csv
3:import json
4:import logging
5:import math
6:import os
7:import shutil
8:import socket
9:import sys
10:import tempfile
11:import threading
12:import uuid
13:import webbrowser
14:from importlib.metadata import PackageNotFoundError
15:from importlib.metadata import version as _pkg_version
16:from urllib.parse import quote
17:import uvicorn
18:from dotenv import load_dotenv
25:from backend.utils.app_home import env_file_to_load, resolve_output_dir
36:from typing import Any, Dict, List, Optional, Tuple
38:from fastapi import FastAPI, HTTPException, Request
39:from fastapi.middleware.cors import CORSMiddleware
40:from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
41:from fastapi.staticfiles import StaticFiles
42:from pydantic import BaseModel
43:from starlette.middleware.trustedhost import TrustedHostMiddleware
45:from backend import (
49:from backend.api import (
52:from backend.config import (
59:from backend.config import ( # new typed config
62:from backend.infrastructure.database import Database
63:from backend.pipeline.segmenters.base import segmenter_registry
64:from backend.pipeline.stitcher.compiler import VideoCompiler
65:from backend.pipeline.stitcher.watermark_i18n import localize_watermark
66:from backend.pipeline.validators.base import registry

```

```

67:from backend.providers import provider_registry
68:from backend.reporting import emit_indexer_report
69:from backend.results import Failure # standardized error/result contract
70:from backend.storage.capabilities import (
73:from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
77:from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
78:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
79:from backend.utils.paths import (
85:from backend.utils.redact import (
88:from backend.utils.single_instance import acquire_lock # P0e: one instance per database
292:from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable on
backend.main)
1484:from backend.api import uploads as uploads_api

```

22. user (2026-07-02T04:48:08.636Z)

```

1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local://<path>             -> local    (local:///abs or local://rel/p)
7          s3://<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs://<bucket>/<key>      -> gcs       (gs:// accepted, normalized to gcs://)
9          gdrive://<path>           -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14
15      from __future__ import annotations
16
17      import re
18      from dataclasses import dataclass
19      from datetime import datetime
20      from typing import Optional
21
22      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
23      _KNOWN = ("local", "s3", "gcs", "gdrive", "gdrive-api")
24
25
26      @dataclass(frozen=True)
27      class StorageStat:
28          """Lightweight object metadata (uniform across backends)."""
29
30          size: int
31          modified: Optional[datetime] = None
32          etag: Optional[str] = None
33          content_type: Optional[str] = None
34
35
36      @dataclass(frozen=True)
37      class StorageRef:
38          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
39          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
40
41          backend: str
42          bucket: str
43          key: str

```

```

44     uri: str
45
46     @property
47     def name(self) -> str:
48         return self.key.rstrip("/").rsplit("/", 1)[-1]
49
50
51     def parse_uri(uri: str) -> StorageRef:
52         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
53
54         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
55         have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
56         """
57         s = str(uri)
58         m = _SCHEME_RE.match(s)
59         if not m:
60             return StorageRef("local", "", s, f"local://{s}")
61         scheme = m.group(1).lower()
62         rest = s[m.end():]
63         if scheme == "gs":
64             scheme = "gcs"
65         if scheme in ("local", "gdrive", "gdrive-api"):
66             # Path-keyed, no bucket: local FS, a path under the mounted Drive, or a path
under a Drive-API
67             # root folder (gdrive-api resolves the path ? a Drive file id).
68             return StorageRef(scheme, "", rest, s)
69         if scheme in _KNOWN:
70             bucket, _, key = rest.partition("/")
71             return StorageRef(scheme, bucket, key, s)
72         raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})")
73
74
75     def resolve_input_uri(
76         raw: str, *, input_backend: str = "local", input_root: str = ""
77     ) -> str:
78         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
79
80         Precedence (first match wins):
81         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
82         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
83         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
84         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
``bucket/x`` ?
85         ``gcs://bucket/x``).
86         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
87
88         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
89         path used today, preserving byte-for-byte behaviour.
90         """
91
92         s = str(raw)
93         if _SCHEME_RE.match(s): # 1. explicit scheme wins
94             return s
95         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
96         # os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
97         # "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".
98         if s.startswith((" /", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):

```

```

99         return s
100     if input_root: # 3. join a relative name under the configured root
101         return input_root.rstrip("/") + "/" + s.lstrip("/")
102     if (
103         input_backend and input_backend != "local"
104     ): # 4. prefix a bucket/key with the declared backend
105         return f"{input_backend}://{s}"
106     return s # 5. bare/relative local path ? unchanged
107

```

23. user (2026-07-02T04:48:14.871Z)

```

44
45     from backend import (
46         billing, # M8: per-job cost ledger (USD internal / INR display)
47         storage, # M2: URI-aware input acquisition (local = identity passthrough)
48     )
49     from backend.api import (
50         auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)
51     )
52     from backend.config import (
53         AppConfig,
54         StartupCheckError,
55         StitchingConfig,
56         enforce_startup_checks,
57         write_light_default_config, # P2b: batteries-included first-run config seeding
58     )
59     from backend.config import ( # new typed config
60         load_config as load_app_config,
61     )
62     from backend.infrastructure.database import Database
63     from backend.pipeline.segmenters.base import segmenter_registry
64     from backend.pipeline.stitcher.compiler import VideoCompiler
65     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
66     from backend.pipeline.validators.base import registry
67     from backend.providers import provider_registry
68     from backend.reporting import emit_indexer_report
69     from backend.results import Failure # standardized error/result contract
70     from backend.storage.capabilities import (
71         storage_capabilities, # P3: secret-free "which media can this box use" report
72     )
73     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
74         ffmpeg_bin,
75         ffprobe_bin,
76     )
77     from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
78     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
79     from backend.utils.paths import (
80         output_base,
81         resolve_under_root,
82         safe_output_filename,
83         validate_input_path,
84     )
85     from backend.utils.redact import (
86         redact_secrets, # P0b: mask secret-shaped fields in /api/config
87     )
88     from backend.utils.single_instance import acquire_lock # P0e: one instance per database
89
90     app = FastAPI(title="Sports Highlight Indexer API")
91
92
93     def create_app(config: AppConfig, db: Database) -> FastAPI:

```

24. user (2026-07-02T04:49:11.091Z)

```

1      """Frame-preserving annotation proxy (STUDIO_MEDIA_LIFECYCLE.md D4).
2
3      A big 4K master is heavy to decode + scrub in a browser. For in-UI annotation we serve a
downscaled
4      proxy that keeps the SAME timeline (fps + duration) ? so a rally marked at t seconds on
the proxy is
5      t seconds on the master, and the labels still key to the master's file MD5. Only
resolution is
6      dropped; duration parity is VERIFIED after the transcode, and a proxy whose timeline
drifted is
7      discarded (the caller falls back to serving the source). Best-effort + stdlib-only;
never raises.
8      """
9      from __future__ import annotations
10
11     import logging
12     import os
13     import subprocess
14     import tempfile
15
16     from backend.utils.ffmpeg import ffmpeg_available, ffmpeg_bin, ffprobe_bin
17     from backend.utils.video import get_video_duration
18
19     logger = logging.getLogger(__name__)
20
21     #: Max proxy height (px). A source already at/below this needs no proxy (serving it is
already light).
22     DEFAULT_MAX_HEIGHT = 720
23     #: Duration must match the source within this (s), else the timeline drifted ? discard
(protects the
24     #: second-based marks the annotator produces).
25     _DURATION_TOLERANCE_S = 0.5
26
27
28     def probe_height(path: str) -> int:
29         """The video's height in px, or 0 on failure."""
30         try:
31             out = subprocess.check_output(
32                 [ffprobe_bin(), "-v", "error", "-select_streams", "v:0", "-show_entries",
33                  "stream=height", "-of", "default=noprint_wrappers=1:nokey=1", path],
34                 text=True,
35             ).strip().splitlines()
36             return int(out[0]) if out and out[0].strip() else 0
37         except Exception as e: # noqa: BLE001 - probe is best-effort
38             logger.warning("proxy: could not probe height for %s: %s", path, e)
39             return 0
40
41
42     def needs_proxy(src: str, max_height: int = DEFAULT_MAX_HEIGHT) -> bool:
43         """True when ``src`` is tall enough that a downscaled proxy is worth generating."""
44         return probe_height(src) > max_height
45
46
47     def _cleanup(path: str) -> None:
48         try:
49             os.remove(path)
50         except OSError:
51             pass
52
53
54     def make_annotation_proxy(src: str, dst: str, max_height: int = DEFAULT_MAX_HEIGHT) ->
bool:
55         """Transcode ``src`` ? a downscaled, timeline-preserving proxy at ``dst`` (written
atomically).
56         Returns True on a verified proxy, False otherwise (caller serves the source). Never

```

```

raises.
57
58     Only the resolution is scaled (`scale=-2:H`); fps/duration are untouched and
duration parity is
59     verified, so second-based marks transfer to the master unchanged. Audio is kept (the
shuttle
60     'thwack' helps the rater judge rally boundaries)."""
61     if not ffmpeg_available():
62         logger.info("proxy: ffmpeg unavailable; serving source (%s)", src)
63         return False
64     if not os.path.isfile(src):
65         return False
66     src_dur = get_video_duration(src)
67     dst_dir = os.path.dirname(dst) or "."
68     os.makedirs(dst_dir, exist_ok=True)
69     fd, tmp = tempfile.mkstemp(dir=dst_dir, prefix=".proxy-", suffix=".mp4")
70     os.close(fd)
71     cmd = [
72         ffmpeg_bin(), "-y", "-i", src,
73         "-vf", f"scale=-2:{max_height}",
74         "-c:v", "libx264", "-preset", "veryfast", "-crf", "28", "-pix_fmt", "yuv420p",
75         "-g", "48", # denser keyframes ? snappy scrubbing
76         "-c:a", "aac", "-b:a", "96k",
77         "-movflags", "+faststart",
78         tmp,
79     ]
80     try:
81         subprocess.run(cmd, check=True, capture_output=True)
82     except Exception as e: # noqa: BLE001 - a transcode failure is non-fatal; fall back
to the source
83         logger.warning("proxy: ffmpeg failed for %s: %s", src, e)
84         _cleanup(tmp)
85         return False
86     proxy_dur = get_video_duration(tmp)
87     if src_dur > 0 and abs(proxy_dur - src_dur) > _DURATION_TOLERANCE_S:
88         logger.warning("proxy: duration parity off for %s (src=%.3fs proxy=%.3fs);
discarding",
89                        src, src_dur, proxy_dur)
90         _cleanup(tmp)
91         return False
92     try:
93         os.replace(tmp, dst)
94     except OSError as e:
95         logger.warning("proxy: could not place proxy at %s: %s", dst, e)
96         _cleanup(tmp)
97         return False
98     logger.info("proxy: wrote annotation proxy %s (%.1fs)", dst, proxy_dur)
99     return True
100

```

25. assistant (2026-07-02T04:50:23.536Z)

Now I have the full picture. Let me check ruff config and how tests run, then set up the worktree.

26. user (2026-07-02T04:50:24.959Z)

```

1     # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2     # bury blame history for zero behavior value).
3     #
4     # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5     # warnings (W) + bugbear (B) + isort (I ? consistent import ordering).
6     target-version = "py38"
7
8     # scratch/ = throwaway debugging scripts that may hit real APIs; it is also

```

```

9      # excluded from pytest collection (see pytest.ini).
10     extend-exclude = ["scratch"]
11
12     [lint]
13     extend-select = ["W", "B", "I"]
14
15     [lint.per-file-ignores]
16     # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
17     # sys.path) before importing backend modules ? import order is load-bearing.
18     "backend/main.py" = ["E402"]
19     "scripts/run_phase3_eval.py" = ["E402"]
20     "scripts/run_process_and_stitch.py" = ["E402"]
21

```

27. user (2026-07-02T04:50:26.071Z)

```

e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
[mypy]

```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade.

Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
```

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

```
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set by create_app() / main()).

Route handlers use request.app.state exclusively; helpers take explicit params.

backend.api.job_worker lifted from quarantine (now takes explicit config/db params).

```
[mypy-backend.main]
```

```
ignore_errors = True
```

Worker dispatcher: composes the SAME validator/segmenter/report building blocks a backend.main, which are typed `config: dict` but receive the AppConfig migration shim at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine until the typed-config migration removes the dict?AppConfig duality everywhere.

```
[mypy-backend.worker.__main__]
```

```
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...])

annotation) ? ratchet those out in a focused typing pass, not here.
(wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

28. assistant (2026-07-02T04:50:33.068Z)

Now let me create the isolated worktree branched from origin/master.

29. user (2026-07-02T04:50:38.181Z)

Preparing worktree (new branch 'fix/api-source-remote-cache')
branch 'fix/api-source-remote-cache' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)

30. user (2026-07-02T04:50:45.626Z)

```
588
589
590     # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels
591     -----
592
593     # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
594     /api/source
595     # don't each kick off a transcode.
596     _proxy_inflight: "set[str]" = set()
597     _proxy_lock = threading.Lock()
598
599     def _proxies_dir(cfg) -> str:
600         return os.path.join(output_base(cfg), "proxies")
601
602
603     def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
604         """Background, best-effort: generate the annotation proxy ONCE so subsequent
605         /api/source loads
606         serve the lighter, seekable copy. Deduped; never blocks the request; the source
607         keeps serving if
608         it fails."""
609         with _proxy_lock:
610             if video_id in _proxy_inflight:
611                 return
612             _proxy_inflight.add(video_id)
613         try:
614             from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
615             if not needs_proxy(src_local):
616                 return # already light enough ? no proxy worth making
617             make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg),
618                 f"{video_id}.mp4"))
619         except Exception as e: # noqa: BLE001 - warming is best-effort
620             logger.warning("proxy: warm failed for %s: %s", video_id, e)
621         finally:
622             with _proxy_lock:
623                 _proxy_inflight.discard(video_id)
624
625     @app.get("/api/source/{video_id}")
626     def get_source(video_id: str, request: Request):
627         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
628         Serves a
629         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
630         still keyed
631         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
```

```

in the
628         background for next time ? non-blocking, so the first load is never delayed. Proxy
warming is
629         LOCAL-source only (remote-source proxying is a follow-up)."""
630         rec = request.app.state.db.get_video(video_id)
631         if not rec or not rec.get("filepath"):
632             raise HTTPException(status_code=404, detail="Video not found.")
633         cfg = request.app.state.config
634
635         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
636         try:
637             proxy_name = safe_output_filename(f"{video_id}.mp4")
638             proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
        _proxies_dir(cfg))
639         except ValueError:
640             proxy_path = ""
641         if proxy_path and os.path.isfile(proxy_path):
642             return FileResponse(proxy_path, media_type="video/mp4")
643
644         # 2) Otherwise resolve + serve the source.
645         try:
646             local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage",
        None))
647             is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage",
        None))
648         except ValueError as e: # unsupported scheme etc. ? a clean client error, never a
500
649             raise HTTPException(status_code=400, detail=str(e)) from e
650         except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
        (not swallowed)
651             raise HTTPException(status_code=502, detail=f"Could not resolve the source
        video: {e}") from e
652         if not os.path.isfile(local):
653             raise HTTPException(status_code=404, detail="Source video file not found.")
654
655         # 3) Warm a proxy in the background for next time (local source only).
656         if is_local:
657             threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
        daemon=True).start()
658
659         media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
660         return FileResponse(local, media_type=media)
661
662

```

31. user (2026-07-02T04:51:22.131Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-source-cache/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

32. assistant (2026-07-02T04:52:09.074Z)

Now the tests. Let me read the worktree copy of the test file, then extend it.

33. user (2026-07-02T04:52:10.179Z)

```

1         """TestClient coverage for the P8 annotate-in-UI engine endpoints:
2         GET /api/source/{video_id} (serve a video's source for annotation) and
3         POST /api/annotations (persist finished rally labels as the byte-verbatim annotator
        CSV)."""
4
5         import pytest
6

```

```

7     pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
8     from fastapi.testclient import TestClient
9
10    import backend.main as m
11    from backend.config.models import AppConfig
12    from backend.infrastructure.database import Database
13
14
15    class _InlineExecutor:
16        def submit(self, fn, *args, **kwargs):
17            fn(*args, **kwargs)
18
19
20    def _client(tmp_path, monkeypatch):
21        db = Database(str(tmp_path / "t.db"))
22        cfg = AppConfig()
23        cfg.output.output_dir = str(tmp_path) # labels land under tmp, not CWD/output
24        monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
25        return TestClient(m.create_app(cfg, db)), db
26
27
28    # ---- GET /api/source/{video_id} ----
29    def test_source_streams_the_recorded_video(tmp_path, monkeypatch):
30        client, db = _client(tmp_path, monkeypatch)

```

34. user (2026-07-02T04:52:20.761Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-source-cache/tests/test_api_annotations.py has been updated successfully. (file state is current in your context ? no need to Read it back)

35. user (2026-07-02T04:52:50.964Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-source-cache/tests/test_api_annotations.py has been updated successfully. (file state is current in your context ? no need to Read it back)

36. user (2026-07-02T04:53:07.785Z)

```

..... [100%]
12 passed in 3.69s

```

37. user (2026-07-02T04:53:39.174Z)

```

C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

```

```

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
81 passed, 4 warnings in 20.37s

```

38. user (2026-07-02T04:53:50.071Z)

```

Exit code 1
B008 Do not perform function call `len` in argument defaults; instead, perform the call within
the function, or read the default from a module-level singleton variable
--> tests\test_api_annotations.py:78:29
|
76 |     """A stat-capable storage stub so the free-space guard can size the object without any

```

```

SDK. """
77 |
78 |     def __init__(self, size=len(_REMOTE)):
79 |         self._size = size
|

```

Found 1 error.

39. user (2026-07-02T04:53:56.829Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-source-cache/tests/test_api_annotations.py has been updated successfully. (file state is current in your context ? no need to Read it back)

40. user (2026-07-02T04:54:49.374Z)

```

All checks passed!
..... [100%]
12 passed in 3.22s
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 2 source files

```

41. assistant (2026-07-02T04:54:55.032Z)

All green. Now verify branch state and commit.

42. user (2026-07-02T04:54:58.274Z)

```

fix/api-source-remote-cache
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
M backend/main.py
M tests/test_api_annotations.py

```

43. user (2026-07-02T04:55:12.825Z)

```

[fix/api-source-remote-cache 924f9a3] fix(api): /api/source downloads a remote source ONCE into
a reusable cache (R1-03)
2 files changed, 225 insertions(+), 9 deletions(-)
924f9a3 fix(api): /api/source downloads a remote source ONCE into a reusable cache (R1-03)

```

44. user (2026-07-02T04:55:24.317Z)

```

origin https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
remote:
remote: Create a pull request for 'fix/api-source-remote-cache' on GitHub by visiting:
remote: https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/api-source-
remote-cache
remote:
branch 'fix/api-source-remote-cache' set up to track 'origin/fix/api-source-remote-cache'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
* [new branch] fix/api-source-remote-cache -> fix/api-source-remote-cache

```

45. user (2026-07-02T04:55:50.889Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/418>

46. assistant (2026-07-02T04:56:02.320Z)


```

+         need = None
+         try: # sizing is best-effort ? a backend that can't stat still fetches (and fails
loudly)
+             need = int(storage.get_storage(ref.backend, config=settings).stat(ref).size)
+         except Exception: # noqa: BLE001 - stat is optional; require_free_bytes(None) is a no-
op
+             need = None
+             require_free_bytes(vid_dir, need, stage="source-cache download")
+             tmp = os.path.join(vid_dir, f".{fname}.{uuid.uuid4().hex}.partial")
+             try:
+                 storage.fetch(ref.uri, tmp, config=settings)
+                 os.replace(tmp, final) # atomic publish ? readers only ever see a COMPLETE file
+             finally:
+                 if os.path.exists(tmp): # fetch (or publish) failed ? never leave a partial behind
+                     try:
+                         os.remove(tmp)
+                     except OSError:
+                         pass
+             return final
+
+
+ @app.get("/api/source/{video_id}")
+ def get_source(video_id: str, request: Request):
+     """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub. Serves
a
+     frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels still
keyed
+     to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy in the
background for next time ? non-blocking, so the first load is never delayed. Proxy warming
is
+     LOCAL-source only (remote-source proxying is a follow-up)."""
+     background for next time ? non-blocking, so the first load is never delayed. A REMOTE
source is
+     downloaded ONCE into ``<output>/source_cache/<video_id>/`` and every later request (each
browser
+     Range request hits this URL anew) streams that local copy ? never a re-download (R1-03)."""
+     rec = request.app.state.db.get_video(video_id)
+     if not rec or not rec.get("filepath"):
+         raise HTTPException(status_code=404, detail="Video not found.")
@@ -641,20 +707,21 @@ def get_source(video_id: str, request: Request):
+     if proxy_path and os.path.isfile(proxy_path):
+         return FileResponse(proxy_path, media_type="video/mp4")
+
+     # 2) Otherwise resolve + serve the source.
+     # 2) Otherwise resolve + serve the source (remote ? the reused source-cache copy).
+     try:
+         local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage", None))
+         is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage", None))
+         except ValueError as e: # unsupported scheme etc. ? a clean client error, never a 500
+         local = _acquire_source_for_serving(video_id, rec["filepath"], cfg)
+         except HTTPException:
+             raise # already a clean client answer (e.g. the 507 free-space guard)
+         except ValueError as e: # unsupported scheme / unsafe id ? a clean client error, never a
500
+             raise HTTPException(status_code=400, detail=str(e)) from e
+         except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced (not
swallowed)
+             raise HTTPException(status_code=502, detail=f"Could not resolve the source video: {e}")
from e
+         if not os.path.isfile(local):
+             raise HTTPException(status_code=404, detail="Source video file not found.")
+
+     # 3) Warm a proxy in the background for next time (local source only).
+     if is_local:
+         threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),

```

```
daemon=True).start()
+   # 3) Warm a proxy in the background for next time. Remote sources now warm too ? the
+   #   source-cache copy is a plain local file (closes the 'remote-source proxying' follow-
+   up).
+   threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()

    media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
    return FileResponse(local, media_type=media)
```

48. assistant (2026-07-02T04:56:23.328Z)

The diff is clean. Let me verify the PR contains only my commit.

49. user (2026-07-02T04:56:27.599Z)

```
{"baseRefName":"master","commits":["924f9a3cd22b697636c4ebd43d3645c2827697ce"],"headRefName":"fix/api-source-remote-cache","number":418,"state":"OPEN"}
```

50. user (2026-07-02T04:56:52.279Z)

Structured output provided successfully